

Deep Learning

Module 1: Introduction & Universal Approximation

Quick Reference Guide

Learning Outcomes:

1. Describe significant impacts and successful use cases of neural networks in contemporary society.
2. Describe the structure and function of a perceptron.
3. Explain how the perceptron uses real-valued inputs for construction of decision boundaries.
4. Explain why multilayer perceptrons are considered to be universal function approximators.
5. Identify the basic properties of a deep neural network.
6. Define a neural network as a function.
7. Describe neural network optimization in terms of function minimization.
8. Define a gradient as a vector of partial derivatives.
9. Explain gradient descent as an approach to minimizing a function.
10. Identify the purpose of a decaying learning rate.

Introduction to Neural Networks

Artificial neural networks have:

- Become the main approach to AI
- Been successfully applied to various pattern recognition, prediction, and analysis problems
- Beaten existing benchmarks
- Solved previously unsolvable problems
- Taken over most AI tasks

The Perceptron

A perceptron is a computation model of a neuron.

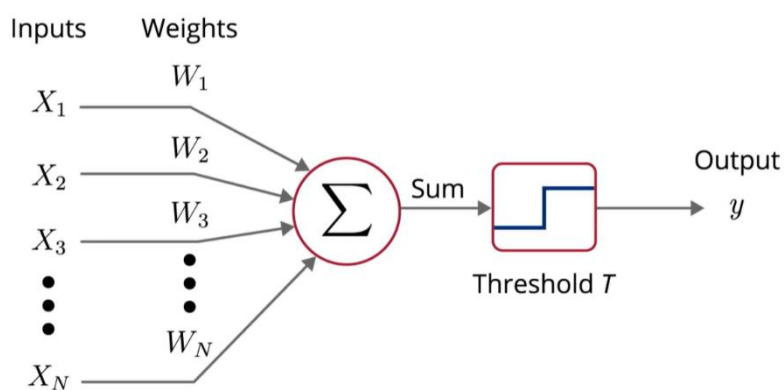


Figure 1: A perceptron with N inputs and weights, a threshold function T , and an output y

The perceptron:

1. Receives multiple inputs with a weight corresponding to each input
2. Computes the weighted sum of the inputs
3. Outputs a 1, if the weighted sum exceeds some threshold T , and a 0 otherwise

Perceptrons can model Boolean gates. Multilayer perceptrons can model arbitrarily complex Boolean functions.

The Perceptron With Real Inputs

Consider real inputs $X_1, X_2, \dots, X_N$ and weights $W_1, W_2, \dots, W_N$.

The output of the perceptron is modeled by:

$$y = \begin{cases} 1; & \text{if } \sum_i X_i W_i - T \geq 0 \\ 0; & \text{otherwise} \end{cases}$$

1. The affine function outputs a weighted sum.
2. The activation function operates on the output of the affine function to output a Boolean value, 1 or 0.
3. The perceptron is said to “fire” when it outputs a 1.

Perceptrons operate on real-valued vector inputs just as linear classifiers do. Linear classifiers operate with linear decision boundaries, which could be a line, three-dimensional plane, or higher-dimensional hyperplane.

We can construct more complex forms of linear decision boundaries by changing the architecture of this class of neural networks.

Consider a perceptron with real inputs X_1 and X_2 and five nodes in the hidden layer.

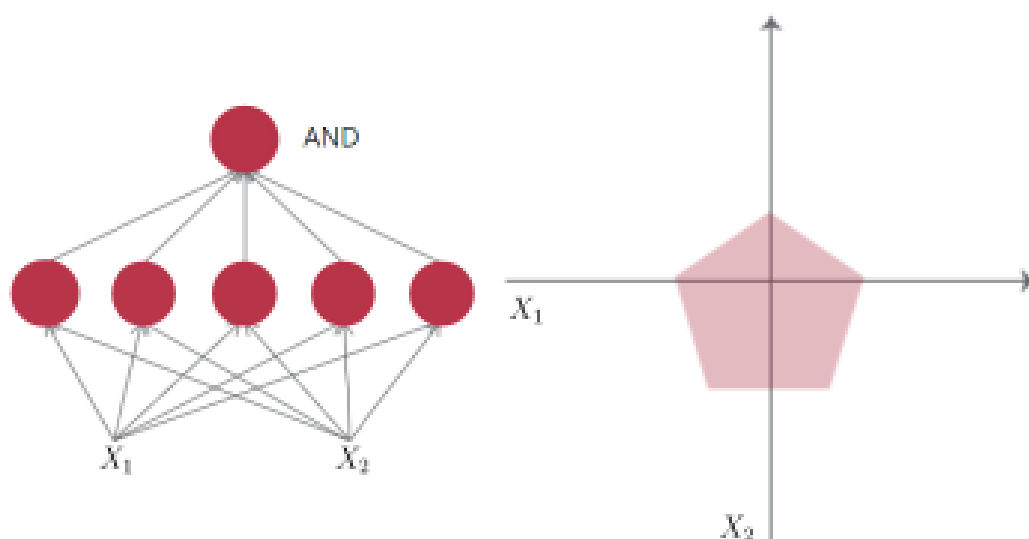


Figure 2: A perceptron with two inputs, five nodes in the hidden layer, and the AND function at the output, and its pentagon-shaped decision boundary.

1. Activate each of the five nodes using the three-component centering formula. This generates overlapping linear decision boundaries or shaded regions in the plane.
2. Process the combined output of the hidden-layer nodes through the AND function to get a pentagon-shaped decision boundary.

Multilayer Perceptrons as Functions

A multilayer perceptron (MLP) is a network of many perceptrons.

MLPs can:

1. Model any decision boundary (universal classifiers)
2. Approximate any scalar function of scalar inputs to arbitrary precision
3. Model any continuous-valued function
4. Approximate any function (universal function approximators)

Multilayer Perceptron Layers

In neural networks, the computation is directed from an input to an output. The depth of a neural network is the length of the longest path from a source to a sink. Any network with a depth greater than two is a deep network.

Multilayer Perceptron Parameters

Functions for voice recognition, image captioning, and game playing can be modeled by neural networks.

The parameters of the perceptron, which determine how it behaves, are its weights and biases.

1. The perceptron computes the affine function, which is the weighted sum of its weights and biases.

2. The output of the affine function is passed through an activation function, which is not necessarily a threshold function.

A bias can be thought of as the weight associated with an extra input whose value is fixed to 1.

How to Learn a Network

Learning a network implies finding the weights and biases such that the network computes the desired function.

Consider a network is a function $f(X, \mathbf{W})$ and a target function $g(X)$.

To learn the network, we must learn the parameters $\mathbf{W}$ such that the error between the computations of $f(X, \mathbf{W})$ and $g(X)$ is minimized.

The divergence function $\text{div}(f(X, \mathbf{W}), g(X))$ quantifies the gap between $f(X, \mathbf{W})$ and $g(X)$.

The error can be defined as the expected divergence, where $P(X)$ is the probability of X .

$$\mathbb{E}[\text{div}(f(X, \mathbf{W}), g(X))] = \int_X \text{div}(f(X, \mathbf{W}), g(X)) P(X) dX$$

The parameters that minimize the expected divergence are denoted by $\hat{\mathbf{W}}$.

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\text{argmax}} \mathbb{E}[\text{div}(f(X, \mathbf{W}), g(X))]$$

However, when the function is not fully specified, computing the integral for the expected divergence will not be possible. An alternative is to sample the function for the known values and compute the average divergence over the training samples or the loss function.

$$\text{Average}[\text{div}(f(X, \mathbf{W}), g(X))] = \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i, \mathbf{W}), d_i)$$

Finally, we minimize the average divergence.

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\text{argmax}} \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i, \mathbf{W}), d_i)$$

A Note About Derivatives

For a function $y = f(\mathbf{x})$, where $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_D \end{bmatrix}$ is a column vector, $\Delta y = \alpha \Delta \mathbf{x}$.

$$\Delta y = [\alpha_1 \cdots \alpha_D] \begin{bmatrix} \Delta x_1 \\ \vdots \\ \Delta x_D \end{bmatrix}$$

$$\Delta y = \alpha_1 \Delta x_1 + \alpha_2 \Delta x_2 + \cdots + \alpha_D \Delta x_D$$

$$\alpha_i = \frac{\partial y}{\partial x_i}; \forall i$$

$$\Delta y = \frac{\partial y}{\partial x_1} \Delta x_1 + \frac{\partial y}{\partial x_2} \Delta x_2 + \dots + \frac{\partial y}{\partial x_D} \Delta x_D$$

$$\Delta y = \nabla_x y \Delta \mathbf{x}$$

$$\nabla_x y = \left[\frac{\partial y}{\partial x_1} \dots \frac{\partial y}{\partial x_D} \right]$$

Therefore, we have defined the gradient as a vector of partial derivatives.

Gradient Descent and Optimization

Gradient descent is an algorithm used to iteratively search for the minimum. Complicated functions, such as the loss function can be minimized through gradient descent.

Gradient descent algorithm:

- Initialize an estimate X_0 .
- Iterate:
 1. Find the direction in which to shift to decrease the error further.
 2. At each iteration, update the estimate by shifting it by a value (step size) that is directly proportional to the negative of the derivative.
 3. Repeat until the estimate can no longer be improved.

In mathematical terms, it can be represented as:

Initialize:

$$x^0$$

$$k = 0$$

Do:

$$x^{k+1} = x^k - \eta \nabla_x f(x^k)^T, \text{ where } \eta \text{ is the learning rate}$$

While:

$$k = 1$$

$$|f(x^{k+1}) - f(x^k)| < \varepsilon_1 \text{ or } \|\nabla_x f(x^k)\| < \varepsilon_2$$

Gradient Descent and Learning Rate

A learning rate beyond which the gradient descent algorithm moves away from the minimum is called a divergent learning rate. We need to start with a large enough learning rate to cause the algorithm to diverge and escape the local minima, and then gradually reduce the learning rate (learning rate decay) with iterations.

Decaying learning rate in neural networks:

1. Train with a fixed learning rate η until the loss stagnates
2. $\eta \leftarrow \alpha\eta$, where $\alpha < 1$ and continue until the loss stagnates
3. Return to step 1 and continue training